



Passec Password Manager

A secure terminal-based password manager that has integrated functions to help improve password security.

Oliver Wright

CMP320: Advanced Ethical Hacking

2024/25

Note that Information contained in this document is for educational purposes.

Abstract

As time goes on, cybercrime only seems to increase. Despite the obvious seriousness of this situation, many companies still seem to lack care in the ways they hold and handle user data. This results in a large amount of data breaches happening every single day even. “Nearly 4000 new cyber attacks occur everyday”, (Palatty, 2025). This is unfortunate for the users as they often suffer the most from companies’ lack of care and can result in them being the victims of identity theft or loss of accounts. Leaked data is often bought and sold behind the scenes, meaning that once an individual's data appears in one data breach, organizations and attackers gain access to it.

While this is a problem, users can take it upon themselves to secure their own data in a few different ways by making it harder to read and use if it does appear in a data breach. This involves having strong secure passwords, making it harder for them to crack if they are the victim of a data breach. They can also regularly change their passwords so that if it is cracked then it is no longer effective when the malicious attacker attempts to use it in criminal activity. This is a little inefficient though, as it would be much better if the individual could change their password as soon as it is breached, preventing attackers from using it.

However, the blame should not fall entirely onto the individual to secure their online identity as, for example, remembering a different strong secure password for each and every service that an individual uses can be very difficult, if not impossible. Therefore, tools need to be developed to tackle this issue and play a simple but vital role in the ever-growing battle between malicious hackers and cybersecurity professionals. This is exactly what Passec Manager intends to be by encouraging proper password practices as well as removing the burden of memorizing each password. The concept of a password manager is something that has been around for some time now, however Passec Password Manager sets itself apart by having built in security checks to inform users if their passwords have somehow been leaked and alerts them, giving them time to change the password before it is used by an attacker.

Contents

1	Introduction	1
1.1	Background.....	1
1.2	Aim.....	2
2	Program Description	3
2.1	Structure Overview.....	3
2.2	Main – “main.py” ()	3
2.2.1	Main() - view figure B1	3
2.3	Database Connection Configuration – “dbconnect.py”	4
2.3.1	Connection Code - <i>view figure B2</i>	4
2.4	User Authentication/Authorization – “userAuth.py”	4
2.4.1	GenVerifyCode() - <i>view figure B3</i>	4
2.4.2	SendEmail() - view figure B3	5
2.4.3	VerifyEmail() - <i>view figure B3</i>	5
2.4.4	RecoverPassword() - <i>view figure B4</i>	5
2.4.5	Register() - view figure B5	6
2.4.6	Login() - view figure B5.....	6
2.5	Post Login Menu (Main Program) – “passwordManager.py”	6
2.5.1	AutoLogout() - view figure B6	6
2.5.2	ResetTimer() - view figure B6.....	6
2.5.3	PasseccManager() - <i>view figure B6</i>	7
2.5.4	GetPass() - view figure B7	7
2.5.5	EditPass() - view figure B7	7
2.5.6	StorePass() - view figure B7.....	8
2.5.7	GenSecCheck() - <i>view figure B7</i>	8
2.6	Program Utilities – “utils.py”	8
2.6.1	HashPass() - view figure B8	8
2.6.2	GenKey() - view figure B8.....	8
2.6.3	LoadKey() - view figure B8.....	8
2.6.4	EncryptData() - <i>view figure B8</i>	9
2.6.5	DecryptData() - <i>view figure B8</i>	9
2.6.6	CheckPwnedPass() - <i>view figure B8</i>	9

2.6.7	CheckPass() - view figure B8	9
2.6.8	CheckAllPass() - view figure B9	9
2.6.9	GeneratePass() - view figure B9	10
2.6.10	CheckPwnedEmail() - view figure B9	10
2.6.11	FormatData() - view figure B9.....	10
2.6.12	GetEmail() - view figure B9	10
2.6.13	StringSearch() - view figure B10.....	11
2.6.14	PassStrengthTester() - view figure B10.....	11
2.7	Other Files	11
2.7.1	.gitignore	11
2.7.2	README.md	11
2.7.3	Procfile	12
2.7.4	Requirements.txt.....	12
2.8	MongoDB Setup.....	12
3	Discussion.....	13
3.1	General Discussion	13
3.2	Future Work.....	14
4	References	16
	Appendices.....	17
	Appendix A - Screenshots	17
	Appendix B – Code.....	22

1 INTRODUCTION

1.1 BACKGROUND

Data breaches are one of the biggest problems in the cybersecurity field. When a company has a data breach it can affect a large number, if not all, of their user base. “The data breach of Yahoo is one of the worst and most infamous cases of a known cyberattack and currently holds the record for the most people affected”, (Chin, 2024). This Yahoo data breach resulted in over 3 billion users’ personal information being leaked over the course of 3 years between 2013-2016. While this is a huge problem, a larger problem is how slow Yahoo was to react in reporting the breach, which meant users were left completely unaware that their data was freely accessible on the internet. If the users were informed of this data breach quicker, then they would have time to change their passwords and reduce the intensity of the breach. However, this is still far from fully negating the potential negative effects.

In 2012, Dropbox had a data breach which affected 68 million users. It’s a classic case of password reuse. “An employee password, stolen during the 2012 LinkedIn breach, was re-used to access his Dropbox account. There, the user had uploaded a document containing Dropbox email addresses, and apparently passwords”, (GHEORGHE, 2016). While this breach is not quite as large as the Yahoo breach, it still had a huge impact on the users whose information was leaked. This could also be much easier prevented seeing as basic strong password practice advises to regularly change passwords and have separate passwords for each different service you use. This can be very difficult to do as a strong password needs to be long and diverse, having a different one for each service would be impossible to remember them all.

Strong password practice dictates that a password should be long and diverse. Microsoft suggests for a password to be strong, “At least 12 characters long but 14 or more is better”, (Microsoft, n.d.). “Cybersecurity experts recommend changing your password every three months”, (McAfee, n.d.). It can be very difficult to follow all these rules for each password especially when users are supposed to have a different password for each service they use. Therefore, this can be solved by having a Password Manager to generate and store strong passwords for the user, locking them all behind one strong password that the user must remember.

1.2 AIM

This project aims to combat bad password habits in a series of different ways:

- Encourage users to have a different password for each service they use, seeing as they do not need to remember them.
- Educate users on what makes a password strong as well as allowing users to generate a strong should they choose.
- Reduce the potential harm that could be caused to a user, should their information appear in a data breach, by searching through data breaches for their leaked information and reporting back if it is found.

2 PROGRAM DESCRIPTION

2.1 STRUCTURE OVERVIEW

The code for this project was separated into a few different files: “main.py”, “dbconnect.py”, “userAuth.py”, “passwordManager.py” and “utils.py”. This makes the code much more readable and means only the necessary functions and libraries need to be imported for each file. It also prevents sensitive data from being accessible from any file; for example, only the “dbconnect.py” file can directly access the database, then the tables are imported to other files and functions.

The program has a database, “PasseManager”, to store user specific details, as there is an account system that makes it accessible to as many users as required. MongoDB was used to host the database, and pyMongo was used to connect to the database from my script using an API key. This database is split into two tables “users” and “passwords”. The “users” table contains account registration and login details such as user_id, username, email address and password. The “passwords” table contains the password manager stored passwords with fields: “password_id”, “user_id”, “service”, “username”, “serPass” and “hashedPass” where “password_id” is the primary key and “user_id” is a foreign key from the “users” table used to identify data for individual users.

The finalized code is uploaded to GitHub to allow users to download it for free from anywhere. API keys and other sensitive data were set as environment variables stored on the host PC to prevent them from also getting updated to GitHub and leaking. This means that users need to set their own environment variables before they can run the program. To make this viable, a README was created and uploaded alongside the other files in the project on GitHub. This details how to set up the project in full.

2.2 MAIN – “MAIN.PY” ()

2.2.1 Main() - view figure B1

The main function works as a pre-login menu screen containing 4 different options: register, login, recover password and exit. This function contains a while loop to ensure the program stays running until the user exits it. The user is prompted with the options and told to provide the number of the option they want. If the user provides an input of non-integer type, then it will be detected as an error and tell the user to input their choice again. After retrieving an input from the user, a match case structure is used to select one of the 4 menu options or a 5th case to handle any integers out with the specified options.

- 1) The first case sends the user to the register function which is held in the userAuth.py file.
- 2) The second case sends the user to the login function which returns the username and userID of the account they are logging into if successful. If the details are incorrect then “None” is returned in place of both the username and userID. An if statement is

then used to check if username has a value other than None, and if it does then the username and userID are set to the `passecManager` function. This is the main password manager loop after authentication.

- 3) The third case allows the user to reset their password in case they have forgotten it and lost access to their account. It returns “true” if it is successful and “false” if it is unsuccessful.
- 4) The fourth case is for exiting the program. Rather than a function, this case simply contains a `break` which will exit the main while loop when run, causing the program to stop running.
- 5) This last case is not for a specific integer, but for any integer out with the specified range of options. All it does is print an error message to let the user know that they selected an invalid option.

A `match` case was used because it is readable and allows for easy error handling if the user inputs an invalid option. It also makes it easy to add new functions in the future seeing as the developed need only to add new integer options.

2.3 DATABASE CONNECTION CONFIGURATION – “DBCONNECT.PY”

2.3.1 Connection Code - *view figure B2*

This file contains no functions leaving all the code unstructured, seeing as it is purely for connecting to the password manager database. MongoDB was used for the programs database and an API key was used to connect to it. This API key was stored as an environment variable to ensure its security when uploading the code openly. The tables are stored into array variables so that they can then be imported into other files when database operations are enacted for login operations or general password manager functions. At the end of the file, a `try` statement is used to attempt to ping the database and return an error if it fails.

2.4 USER AUTHENTICATION/AUTHORIZATION – “USERAUTH.PY”

This file imports the “users” table to perform login and registration functions. The file also needs to import `smtplib` and `ssl` to send emails to the users. This happens when the user is registering their account and needs to verify their email, or when they have forgotten their password and need to reset it. The `random` library is also imported so that codes can be randomly generated and sent in these emails. Lastly, the file imports the `os` library so that the Passec Password Manager’s official email address and email address password can be stored as environment variables instead of being visible.

2.4.1 `GenVerifyCode()` - *view figure B3*

When either verifying the email address that the user is trying to register with, or sending a password reset verification code, this function is used to generate the 6-digit code that is sent to the user’s email address. An empty string variable “code” is initialized. The function then loops for 6 iterations, each time using the `random` library to pick a random number between 48-57. This is the `ascii` value range for the number 0-9. The randomly picked number is appended onto the “code” variable. After exiting the loop, the function returns the code.

2.4.2 SendEmail() - view figure B3

This function handles the process of sending the email. It takes in the user's email address that it is sending to, and a randomly generated code as parameters. First the message logic is handled, setting up what the email is going to contain. Then default SSL context is established in a variable (this is required for the command that officially establishes a connection with the gmail SMTP server). The try and except built-in functions are used to handle errors with sending mail. The function now uses the with command when establishing the connection, which ensures that the connection is properly closed after performing the required actions. "smtplib.SMTP_SSL" is used to establish the connection. This function takes in parameters "host, port, context" which are respectively set as "smtp.gmail.com, 465, context=context". This uses gmail's SMTP (simple mail transfer protocol) server as well as the default SMTP port over SSL. The context is set to default; however, this could be used for configuring different aspects of the secure connection. The function then logs into the SMTP server using the information of the gmail account that was set up to send out the emails. The email is then sent to the user.

2.4.3 VerifyEmail() - view figure B3

This is called from the "register()" function, and its main purpose is to prevent people from creating accounts with another person's email address. It takes in the user's email address as a parameter. First, it calls the "genVerifyCode()" function to generate a code to send to the email. It then sends the user's email and the code as parameters to the "sendEmail()" function to send the email. Following this, it uses a while loop to allow the user 3 attempts at inputting the correct code. The function receives an input from the user for each iteration of the loop. If the code the user inputs is equal to the code generated and sent to the email address they entered, then the function returns true. If not, the attempts counter is incremented by 1. If the function exits the while loop, then it means the user did not enter the correct code by the third attempt and it returns false.

2.4.4 RecoverPassword() - view figure B4

An input is received by the function to get the user's email address. The "users" table is then queried for this email address, and if it is not found, then the function returns false. A code is then generated using the "genVerifyCode()" function and the "sendMail()" function is used to send this code to the user's email address. The user then gets 3 attempts to input the code the exact same way as the "verifyEmail()" function does – a while loop counting the users attempts and an if statement comparing the user input to the code. If the code is correct (and it enters inside the if statement), a string called password is initialized and assigned "None". The function then starts another while loop checking while password is "None". Inside this loop, an input is retrieved from the user of what they want their new password to be (the user is also offered the option to leave it blank and have the password manager generate a strong password for them). An if statement is used to check if the password is blank, if it is then the "generatePass()" function is called which returns a strong randomly generated password. The "passStrengthTester()" function is then used to check if the password is strong enough; if this function returns false then the password is too weak, and it repeats the while loop. If the "passStrengthTester()" function returns true, then the initially created variable "password" is set to equal the user's new password, and it exits the while loop as "password" no longer is "None". After that, the users table is updated to give this user their new password (using the user's email address as a unique identifier). The function then returns true. If the function had reached 3 attempts and still not had the correct code entered, then it would have exited the first while loop, which would then skip all the previously talked about logic in this paragraph. In such a case, the function would then return false.

2.4.5 Register() - view figure B5

It was decided that 3 fields would be required for the registration process: username, email address, and password. Three separate variables are initialized to hold these inputs, each initially given the value None. Three separate while loops are used to get valid inputs for each field, this code structure prevents the user from needing to retype every field if just one is invalid. The username field only requires it to be unique to other users' username on the password manager system. The email address has the same requirement as the username, forcing it to be unique; however, the email address must also be verified. This is done by sending the email to the "verifyEmail()" function and if it returns true then the email is set. The password field, however, is a little more complicated. The user is offered the option to leave the password field blank and therefore have the system generate a strong password for them. This is done by another function held in the utilities file "utils.py" which is talked about later. The password, whether it is generated or not, is then sent through another function "passStrengthTester()" which checks whether the password follows the password manager's password policy. This function is also talked about later. When all three fields (username, email address, and password) are valid, the user's account is registered, and the details are stored in the users table in the database. Before the password is stored in the database, it is sent to a hashing function "hashPass()" which hashes it using the sha256 algorithm. This makes it less detrimental if there were somehow a data breach as the passwords would not be in plaintext form.

2.4.6 Login() - view figure B5

The login function requires only the user's username and password to login. After the user inputs both their username and password, a query is sent to the database to check if there is an account that matches the given username. If the account exists, then the password the user input is hashed and compared to the hashed password stored in the database. If the details match, then the function returns the user's username and userID values. If the details do not match, then the login fails, and it returns "None" for username and userID.

2.5 POST LOGIN MENU (MAIN PROGRAM) – "PASSWORDMANAGER.PY"

This file imports the usersTable and passwordsTable so that it can be worked with in later functions. It also imports the "time" and "threading" libraries and has some global variables initialized in order to have an automatic inactivity logout function.

2.5.1 AutoLogout() - view figure B6

This function handles the logging out if it completes fully. It starts by initializing the global variable "loggedIn" flag, then it uses "time.sleep(idleTime)" to sleep for the set 2 minutes. After this it uses an if statement to check if the "loggedIn" flag is true and if it is then it sets the flag to false which returns the user to the "main()" function. The thread running the function is destroyed and reinitialized every time the "resetTimer()" function is called.

2.5.2 ResetTimer() - view figure B6

First the function calls the global "logoutTimer" variable to check if there is already a timer running, if there is then it cancels it and creates a new timer (essentially resetting it). It uses threading to run the "autoLogout()" function which handles logging the user out if it manages to fully complete. This whole

system relies on user inputs cancelling the function from fully finishing which takes however long the variable “idleTime” is set to. The variable is set to 120 which is equivalent to 2 minutes.

2.5.3 PassecManager() - view figure B6

After a successful login, the passecManager function is called, sending both the user’s username and userID. This function is structured very similarly to the main() function as it has a while loop to remain here until the user logs out. The main operation in this while loop is to start the inactivity timer by calling the “resetTimer()” function. It also has a match case menu like the “main()” function with the same error handling method checking for invalid data type or a value out with the specified options. Every time the program receives an input from the user it calls the “resetTimer()” function, restarting the inactivity counter. This menu has 7 options:

1. getPass() – Retrieve all the users stored passwords (excluding their master password).
2. storePass() – Store a new password to the users stored password list.
3. editPass() – Edit a previously stored password.
4. checkPwnedEmail() – Scrape the web for instances of the users email address appearing in data breaches.
5. checkPwnedPass() – Scrape the web for instances the user’s master password appearing in data breaches.
6. genSecCheck() – A general security check that scrapes the web for instances of any of their stored passwords appearing in data breaches.
7. Logout – here instead of having a function, the built-in python break function is called to exit out of the while loop and send the user back to the pre-login screen.

2.5.4 GetPass() - view figure B7

This function takes in the user’s userID. The passwords table is queried using userID to get all their passwords. The program then loops through the array and sends each password to the decryptData() function in the “utils.py” file. After decrypting the password, it is then printed with the username and service that it was stored alongside.

2.5.5 EditPass() - view figure B7

The user’s userID is taken into the function so that it can be used in a database query later. The user is asked for an input of the name of the service that they would like to edit their password for. The database is then queried to return the record (if it exists) of the service the user input alongside their userID. An if statement then checks if a record of this exists or if the user’s input is empty. If this is true, then the function returns false. If not, then the user is prompted for a new input to get what they want the new password to be, they are also given the choice to have a password generated by leaving the input empty. An if statement checks whether the input is empty and if it is it sends it to the generatePass function in “utils.py” to generate a strong 16-character password which complies the master password’s password policy. Two instances of the plaintext password, whether it is user input or generated, are then encrypted and hashed respectively. This is so that it can be safely sent to the HIBP (have I been pwned) API to check data breaches for it without sharing the plaintext password or the full hash. Storing the hashed plaintext password is also vital to ensure that the storePass() function works properly. The record is then updated in the passwords table and the function returns true.

2.5.6 StorePass() - view figure B7

The user's userID is also taken into this function so that it can store the password under the correct user in the database. The program needs three different values to store a password: service, username and password. After getting these inputs, the password is hashed using the hashPass() function from "utils.py". The database is then queried to search for a record of this already existing and if this is found then the function returns false. This prevents the database from storing duplicate records. If the query does not find any duplicate records, then the password is encrypted, and all the data is inserted into the passwords table. Lastly, the function returns true.

2.5.7 GenSecCheck() - view figure B7

Like the other functions in the file, this function also takes in the user's userID. The passwords table is then queried to return all records belonging to the user using the userID. These records are stored in an array. The function loops through the array decrypting the encrypted passwords using the decryptPass() function from the "utils.py" file and storing this in a variable. It also stores the service of this record in its own variable too. The checkPwnedPass() function from "utils.py" is used on the plaintext password to check for instances of it appearing in data breaches. Then the password is sent through the passStrengthTester() function from "utils.py" to check its strength. If this function returns true, then the function prints that the password is strong. If the function does not return true, then the passwords strength flaws are printed out in the passStrengthTester() function rather than in this function.

2.6 PROGRAM UTILITIES – "UTILS.PY"

2.6.1 HashPass() - view figure B8

This is a simple function that takes in a password and returns the password hashed using the sha256 hashing algorithm using the imported hashlib library. The password must be encoded UTF-8 before it can be hashed in sha256. The hashing algorithm returns a binary value and therefore the built-in python hexdigest() function must be used to convert it into a readable hexadecimal string.

2.6.2 GenKey() - view figure B8

This is a one-time use function which was used to generate the encryption key for the passwords stored in the passwords table. The cryptography library is imported in order to generate the key. The Fernet.generate_key() function is used to generate the key and store it in a file called "secret.key". Originally the key was just called from this file, however, this was not secure for the final project as people would be able to access and read this key when uploading it to github. Therefore, the secret key was set as an environment variable on the local operating system. This means that the same environment variable had to be set on the cloud server for the programs online services to function properly.

2.6.3 LoadKey() - view figure B8

The loadKey() function simply reads the environment variable from the operating system. After attempting to read the variable it does an if statement to check if the variable exists to handle errors. If the key does not exist then it returns none, if it does exist, then it returns the encryption key. This key is used in both the encryptData() function and the decryptData() function.

2.6.4 EncryptData() - view figure B8

This is a one-line function that is used to encrypt the user's passwords using the encryption key previously generated before storing them in the passwords table of the database. The plaintext password must be encoded in order for the cipher.encrypt function to work. After encrypting the data using the cipher library, the encrypted password is decoded so that it can be returned and stored in the database via the other functions that use this one.

2.6.5 DecryptData() - view figure B8

The decryptData() function works almost entirely the same as the encryptData() function. Much like its counterpart, the function needs to encode the password before it can perform its operation, then can decode it to get the readable result. The only difference here is that the function is fed encrypted passwords and decrypts them using the decrypt function of the cipher library.

2.6.6 CheckPwnedPass() - view figure B8

A password is fed to this function via two separate methods, the Passec Manager main menu option to search for the user's master password, or the genSecCheck() function which feeds all the user's stored passwords into it. The checkPwnedPass() function makes use of the HIBP (have I been pwned) API to search for instances of passwords appearing in data breaches. This requires great care to do as simply feeding plaintext passwords to a 3rd party API would be inadequate security practice therefore there is a set method to do this safely. The plaintext password is hashed using the sha1 algorithm, seeing as this is what the HIBP API allows. A prefix and a suffix are taken of the hashed password, the first 5 characters and the remaining 35 characters. The hash prefix is sent to the HIBP API which returns the remaining 35 characters of all passwords found in data breaches beginning with that hash prefix as well as the number of times that password has appeared in a data breach. The data is handled by splitting each line and storing the parts into an array. When looping through this array, the hash and number of times the password has appeared in a data breach are stored in separate temporary variables respectively. An if statement is carried out to check if the user's password's hash suffix matches the current array hash suffix, if it does then the function returns a string detailing the number of data breaches that the password has appeared in. The function loops through this array until the password is found or all hashes are checked, and if nothing is found it means that the password was not found in any data breaches.

2.6.7 CheckPass() - view figure B8

Since the user's master password cannot be stored in the database to ensure security, the user is asked to retype when wanting to perform a security check on it. The checkPass() function is vital as it allows the program to check if it is the correct password, and then safely send it to the HIBP API without needing to store it in plain text or encrypted form. It works the same as a simple login function which means it takes in the username and password that it is checking. It queries the users table for the record belonging to this user. After that it checks if the hashed password attached to this record matches the hash of the password the user just input. If it matches then the function returns true, if it does not match then the function returns false.

2.6.8 CheckAllPass() - view figure B9

This function is to check data breaches for instances of any of the user's stored passwords appearing. The user's userID is taken into this function so that the passwords table can be queried to bring up all the users stored password records, and they are stored in an array. A for loop is used to loop through

the array, decrypting the password using the `decryptData()` function and then checking data breaches for the current password using the `checkPwnedPass()` function. The results are printed out each iteration of the loop.

2.6.9 `GeneratePass()` - view figure B9

If the user decides not to insert a password for one of the services, then a password is automatically generated by this function. The function takes in a password length as a parameter which is set to 16 by default. The password variable is initialized as an empty string so that a while loop can run until the password is strong. It loops through the password length number of times adding a random character within a certain ascii range each iteration. This range contains uppercase characters, lowercase characters, numbers and special characters which are all required for the password to be deemed strong. The generated password is then sent through the `passStrengthTester()` function, if this function deems it not strong enough (returns false) then password is set to an empty string again and a completely new one is generated. When a strong enough password is generated, the password is returned by the function.

2.6.10 `CheckPwnedEmail()` - view figure B9

This function performs the same task as the `checkPwnedPass()` function, except it checks for the user's email address appearing in data breaches instead. First the function takes in the user's email as a parameter and hashes it using the sha256 algorithm. This way the email is not being sent to the API in plaintext. The hash is sent off to the leakcheck API using the requests library. If the program fails to connect to the API, then the error message is printed. If the program successfully gets a response in the form of a JSON response "response.json" which is then sent to the `formatData()` function.

2.6.11 `FormatData()` - view figure B9

This function handles the information given in response to the `checkPwnedEmail()` function as this can contain lots of raw data. This data is split into a variable "varFound" and an array "source". Two empty arrays are initialized "nameArr" and "dateArr" to split up the data in the sources array. A loop is used to loop through the sources array and separate the name of the site the breach was found on from the date it was leaked. Storing this data in separate arrays makes it very easy to work with and match the data with the right counterpart because iteration "x" of one array's counterpart will always be iteration "x" of the other array. An if statement is used to check if any data was found using the "varFound" variable and if it was then a loop is used to print out all the data using the two arrays created. If no breaches are found, then a print statement is issued to inform the user of this.

2.6.12 `GetEmail()` - view figure B9

Seeing as the program does not currently require an email nor does it carry the email address through as a variable after login, the database must be queried to check and retrieve the user's email address if it exists. This function does precisely that, it queries the users table using the user's userID to return the email address attached to their account (if it exists). If the email address exists then the function returns it, if it does not exist then the function returns "None". After the function returns a value to the `passecMain()` function, an if statement is used to see if an actual email address was returned and if it was then it continues to the `checkPwnedEmail()` function.

2.6.13 StringSearch() - view figure B10

This function is used in the `passStrengthTester()` function to check that the password fed into it contains at least one special character and at least one uppercase character. A string `"spChar"` is initialized to store all the special characters that the function should search for. Two other variables were initialized and set to false, these variables are flags that are to be set to true when a special character is found and when an uppercase character is found respectively. The function loops through each character of the password first checking if the password contains an uppercase character and setting the `"uppercaseFlag"` to true if it does. A nested loop is then set to check if the current character of the password matches any of the characters stored in the `"spChar"` string variable and if it does then the `"spFlag"` is set to true. The `"uppercaseFlag"` and `"spFlag"` are then returned.

2.6.14 PassStrengthTester() - view figure B10

This is where the passwords strength is tested according to a few different rules. This function is structured by setting a variable `"passIsValid"` to true and then performing a few checks to see if it follows the predefined rules and setting this variable to false if one or more of the rules are not followed. The function sends the password to the `stringSearch()` function and retrieves the `"uppercaseFlag"` and `"spFlag"` from it. The first check that is performed is that the password is no less than 8 characters. The second check is whether the `"uppercaseFlag"` is true or false. The third and final check is whether the `"spFlag"` is true or false. If any of these requirements are not met, then a message is printed out to explain the current problem and `"passIsValid"` is set to false. The function then returns `"passIsValid"`.

2.7 OTHER FILES

2.7.1 .gitignore

This is a github configuration file that prevents certain files/file types from being uploaded to the github page. This is useful for files containing sensitive information. Currently this file contains `"secret.key"`, `"__pycache__/"` and `".env"`. This prevents any keys, files contained within the cache directory or environment files being uploaded to GitHub. While none of these files currently exist in the program (other than the `pycache` directory), they were used throughout the development and testing phases of the program. It is also good to keep sensitive file types and directories in here to prevent leaking sensitive data after future development of the program.

2.7.2 README.md

The `"README.md"` file was created to accurately and efficiently describe the purpose of my program as well as give in-depth guidance on how setup and use it. The markdown (`".md"`) file format was chosen as it is the most common for modern README files and allows much nicer formatting than a basic text file format. The markdown code for this file can be seen in *figures (B11 & B12)*. The file describes how to set up the program and can be summarized as:

1. Setup database
2. Setup verifying gmail account
3. Generate encryption key
4. Set environment variables
5. Run

While the terminal-based menu system is quite self-explanatory, this file attempts to give further insight into what each option does to ensure the user is performing the task they are trying to. The result for this code is shown in *figures (A6 – A11)*.

2.7.3 Procfile

This file was required by DigitalOcean to help it understand how and what to run on startup. This file will be much more useful when a web application has been developed for Passec Password Manager and will be an essential component of its successful functionality.

2.7.4 Requirements.txt

The “requirements.txt” file was also required by DigitalOcean to install all the required python dependencies for running the program on the cloud. It contains 6 different dependencies as well as a version type for each required to successfully run the program.

2.8 MONGODB SETUP

A mongoDB cluster was created to hold the Passec Password Manager database. Inside this cluster, a database was created and named “PassecManager”. Two tables were created inside this database. The “users” table contains all the user created accounts on Passec Password Manager and the “passwords” table contains all the passwords stored on these accounts. The fields do not need to be specified as the mongoDB automatically adds fields when records are inserted into the tables from the python script.

3 DISCUSSION

3.1 GENERAL DISCUSSION

There are currently two instances where the program makes use of environment variables. This is when the variable is stored in a specific file on the operating system. The program can call these variables from outside the project files and therefore prevents users from seeing them. This is important because the code is open source. It also helps hide these variables from attackers in a scenario where they were able to gain access to the cloud server. The two environment variables are the HIBP (have I been pwned) URI/ API key and the encryption key used to encrypt and decrypt the passwords stored in the database (excluding the master password).

The password manager also explains what creates a strong password and enforces it for the master password at least. A strong password should be at least 12 characters long and include at least one of each character type: uppercase, lowercase, special (!?@#%) and lastly numerical. Something to consider is that the password manager could enforce that the stored users stored passwords also follow this password policy to increase their overall online security. However, some people simply refuse to do this despite knowing about their poor online security, therefore the application allows them to store weaker passwords while still encouraging better practices. There is also a small chance that other services disallow users to create strong passwords according to the standards set by Passec Password Manager, seeing as even many companies are not up to date on password security.

Much care needs to be taken when using APIs that check for data breaches, because if the data is not handled correctly then this function alone could result in a breach of data. For checking for pwned passwords (passwords that have appeared in data breaches), the HIBP (have I been pwned) API is used. The password that is being checked is sent to the API and then the number of times that it is found in data breaches is returned. To securely transfer passwords to the API to check, a specific methodology was used. The password is first hashed in an algorithm that the HIBP API accepts (sha1), so that the plaintext password has nothing to do with the API. Then the first 5 characters of the hash are sent to the API, which returns all hashes found in data breaches (as well as the results for each) that start with these first 5 characters. After that, the correct hash is found by matching the remaining 35-characters of the hash with the results returned. When checking for if the user's email address has appeared in a data breach, the leakcheck API is used. To securely send the email address to the API, it is first hashed in sha256. After retrieving it, the API returns the number of times it is found in data breaches as well as the services that it was found on. Another function was required to handle this data as it can return lots of information.

Something that had to be considered is the possibility of a user leaving their device unattended while logged into Passec Manager. While it is mainly up to the user to avoid doing this, Passec Manager still attempts to take precautions against it. To combat this, an auto logout function was implemented which logs the user out of their account if they are inactive for a 2-minute period. This is important as if an unauthorized person managed to access a user's account while they were logged in, they could gain access to all the user passwords for other services.

Overall, Passec Manager succeeds in improving general password security. The program encourages users to not only use strong passwords but also use a different unique password for each service they sign up for. This means that if the user's data appears in a data breach, it is likely that there will only be one password appearing. Seeing as Passec Manager scrapes the web for passwords appearing in data breaches, the software can alert the user and encourage them to change it before it can be used against them maliciously. Should the user's hashed passwords appear in a data breach, Passec Manager's encouragement for strong passwords, which are long in length and contain a wide variety of character types, prevents the likelihood of the user's leaked hashes being cracked by a brute-force attack.

3.2 FUTURE WORK

A solution was never worked out in terms of setting up the cloud server. Input handling was overlooked when developing the program and therefore causes the program to crash when requesting a user input on the cloud server. This prevents the possibility of accessing passwords from other devices unless the user has the software and same encryption key on all devices. This is unideal as the setup for the program is not currently as optimal as it could be.

The program makes use of environment variables for the MongoDB URI and the encryption key used on the passwords. This means that the software would not work unless the device running it has this environment variable. This is fine for cloud functionality because the environment key can be set on the cloud server; however, if local software wanted to be created and distributed, then an alternative solution would need to be developed.

As of now, the program has one encryption key for all the passwords; however, each user could have their own encryption key generated on their device for extra security. That means only the user will be able to decrypt their stored passwords if they are logged in on the device. Another solution would then have to be created for safely transferring this encryption key to other devices that the user wanted to access the program from. It would also have to allow the cloud to use their encryption key otherwise that will no longer work.

A GUI (graphical user interface) could also be useful to make it easier for users to interact with the program, seeing as a command terminal is not as user friendly. This would also make cloud integration much easier, as it could receive inputs directly from a web application hosting the GUI. Instead of inputting values and strings to traverse the menu, the web application could make use of forms and interactive buttons to different webpages. It would also allow for much cleaner output for large bodies of text, for example, the breach scan results.

Another feature that should be added is multi-factor authentication. This is where a user must verify that it is them trying to login from another device belonging to them. This will make the program much more secure, as if an unauthorized person manages to get the user's master password, then they have access to all of the user's passwords. Multi-factor authentication could also be assigned to specific features as well to improve overall account security. This could include register, login, reset password, password retrieval, password editing, and lastly the security check functions.

4 REFERENCES

- Chin, K. (2024, December 30). *Biggest Data Breaches in US History (Updated 2025)*. Retrieved from UpGuard: <http://upguard.com/blog/biggest-data-breaches-us>
- GHEORGHE, A. (2016, August 31). *Massive hack alert! 68 million Dropbox credentials leaked online*. Retrieved from Bitdefender: <https://www.bitdefender.com/en-gb/blog/hotforsecurity/massive-hack-alert-68-million-dropbox-credentials-leaked-online>
- McAfee. (n.d.). *How Often Should You Change Your Passwords?* Retrieved from McAfee: <https://www.mcafee.com/learn/how-often-should-you-change-your-passwords/#:~:text=But%20how%20often%20should%20you,has%20access%20to%20your%20account.>
- Microsoft. (n.d.). *Create and use strong passwords*. Retrieved from Microsoft: <https://support.microsoft.com/en-gb/windows/create-and-use-strong-passwords-c5cebb49-8c53-4f5e-2bc4-fe357ca048eb#:~:text=Create%20strong%20passwords&text=A%20strong%20password%20is%3A,character%2C%20product%2C%20or%20organization.>
- Palatty, N. J. (2025, February 15). *How Many Cyber Attacks Per Day: The Latest Stats and Impacts in 2025*. Retrieved from getastra: <https://www.getastra.com/blog/security-audit/how-many-cyber-attacks-per-day/#:~:text=How%20Many%20Cyber%20Attacks%20Happen,malware%20are%20detected%20every%20day.>

APPENDIX A - SCREENSHOTS

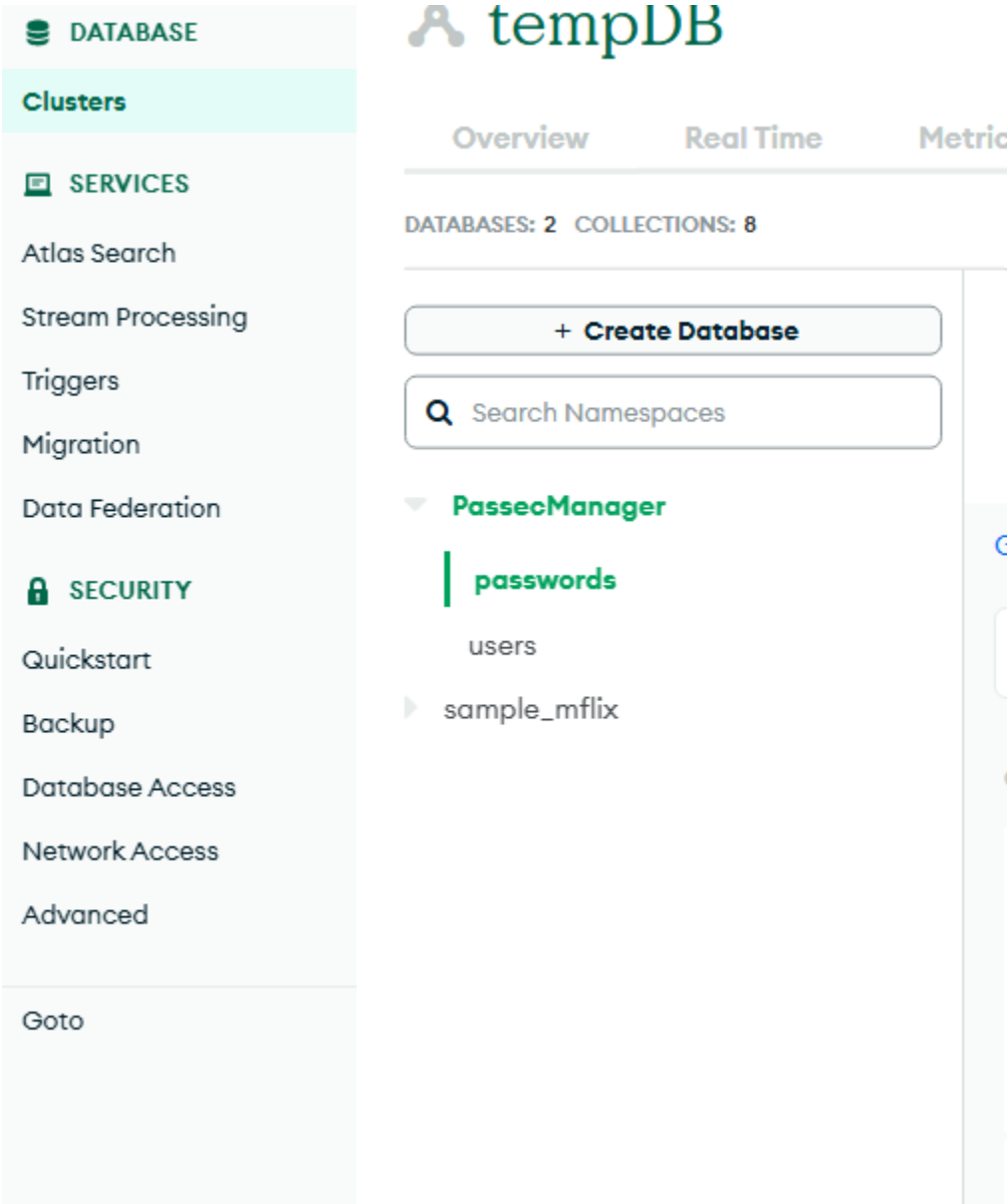


Figure A1 - MongoDB cluster containing database and tables

Connect to tempDB



Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js ▼	6.7 or later ▼

2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://admin:<db_password>
```



Replace `<db_password>` with the password for the **admin** database user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)

[Node.js Starter Sample App](#)

[Access your Database Users](#)

[Troubleshoot Connections](#)

Figure A2 - MongoDB database connection URI

```
(kali@kali)-[~]
$ ssh root@[REDACTED]
The authenticity of host '[REDACTED] ([REDACTED])' can't be established.
[REDACTED] key fingerprint is [REDACTED].
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[REDACTED]' ([REDACTED]) to the list of known hosts.
root@[REDACTED]:~# password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-9-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr  2 20:01:24 UTC 2025
```

Figure A3 - SSH connection to digital ocean cloud droplet

```
export mongodbURI=[REDACTED]
```

Figure A4 - Environment variable for mongodb URI

```
root@ubuntu-s-1vcpu-1gb-lon1-01:~# echo $SHELL
/bin/bash
root@ubuntu-s-1vcpu-1gb-lon1-01:~# nano ~/.bashrc
root@ubuntu-s-1vcpu-1gb-lon1-01:~# echo $mongodbURI
[REDACTED]
root@ubuntu-s-1vcpu-1gb-lon1-01:~# source ~/.bashrc
root@ubuntu-s-1vcpu-1gb-lon1-01:~# echo $mongodbURI
[REDACTED]
```

Figure A5 - Setting mongodbURI API key environmental variable on cloud server

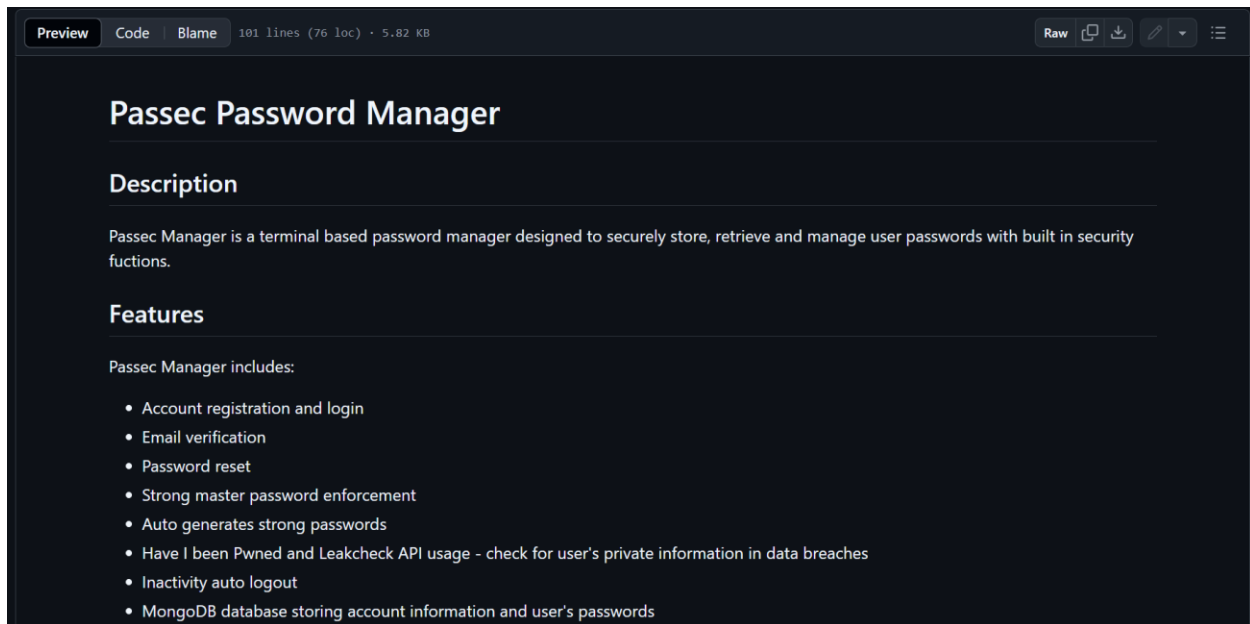


Figure A6- README graphic (1/6)

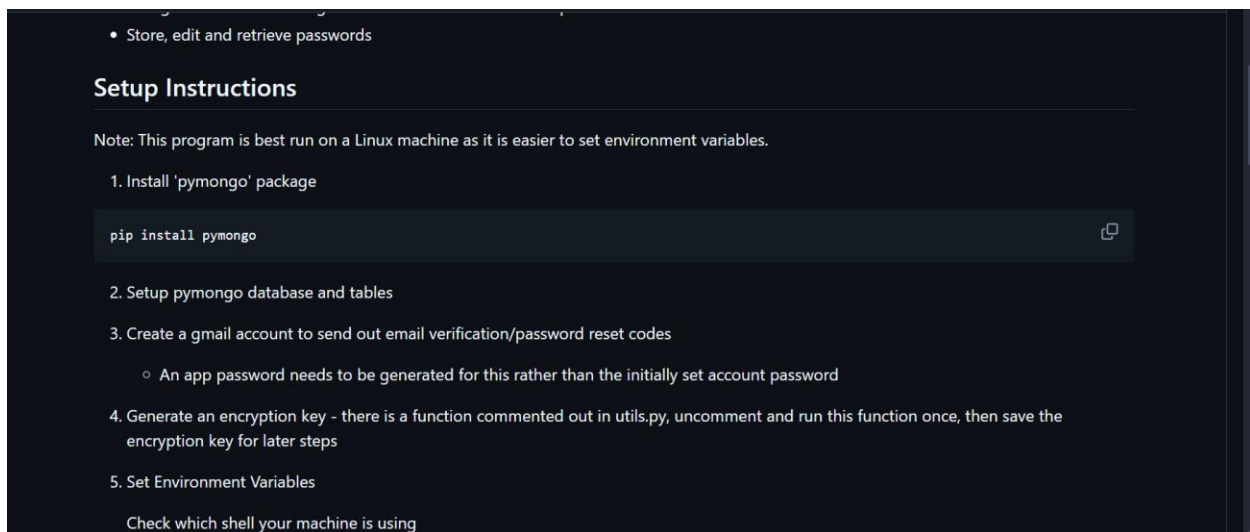


Figure A7 - README graphic (2/6)

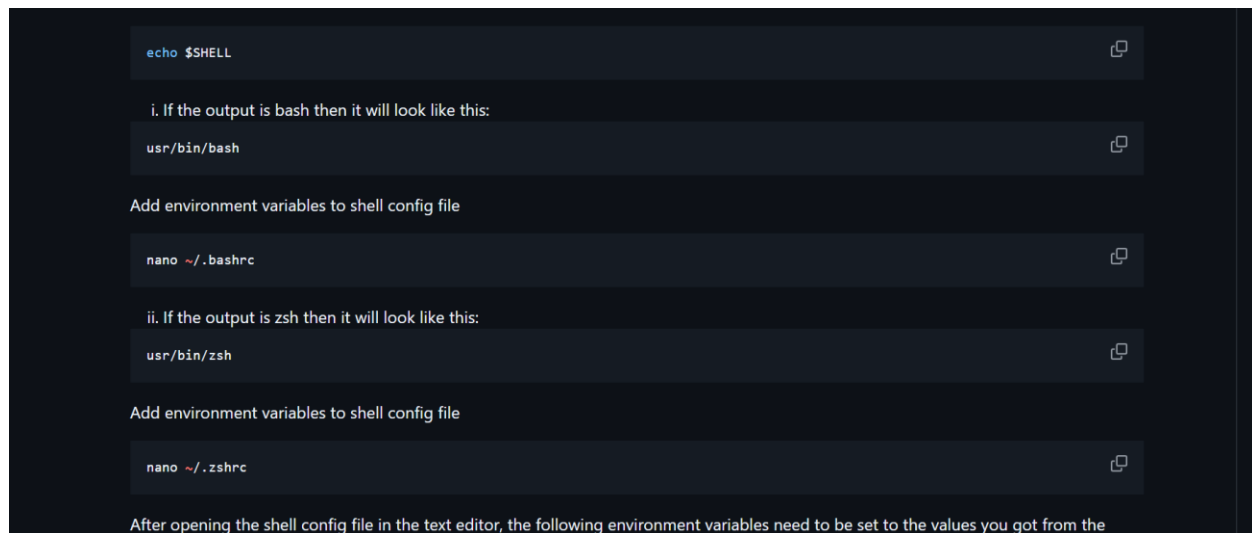


Figure A8 – README graphic (3/6)

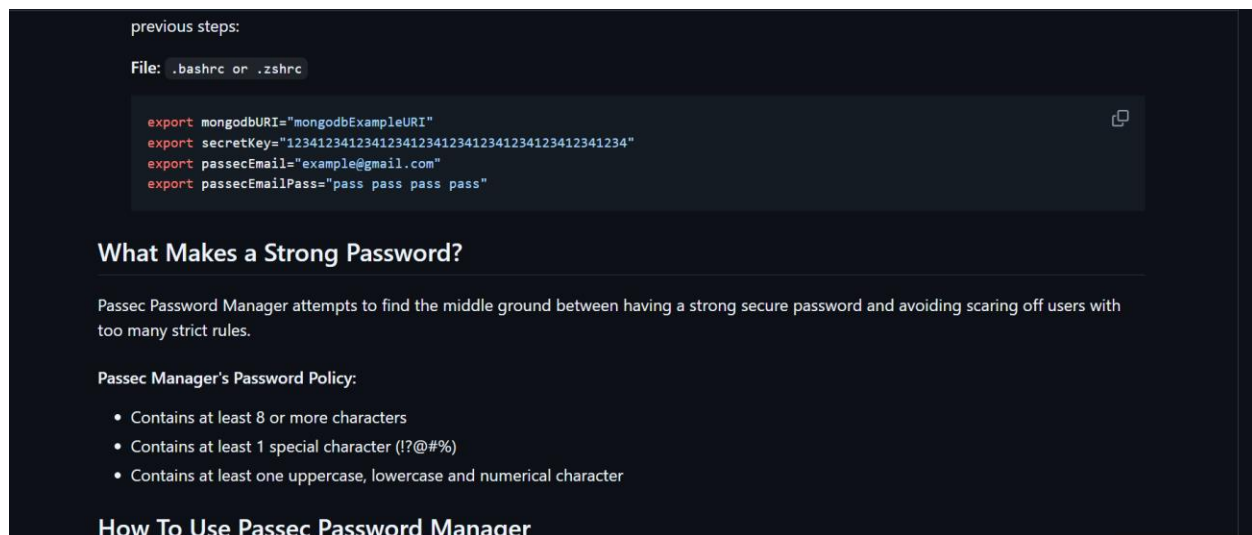


Figure A9 – README graphic (4/6)

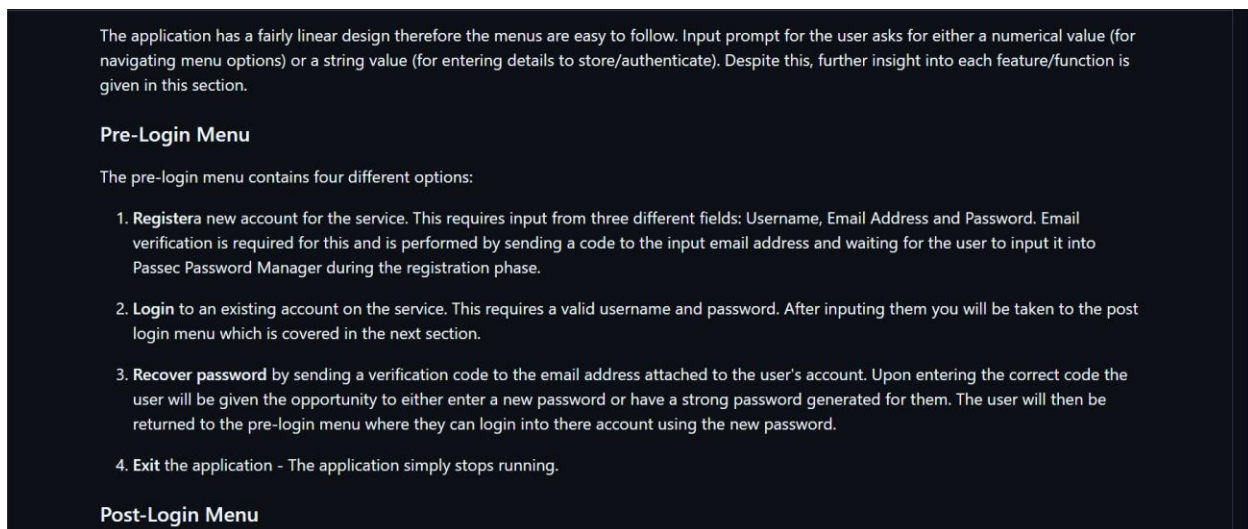


Figure A10 – README graphic (5/6)

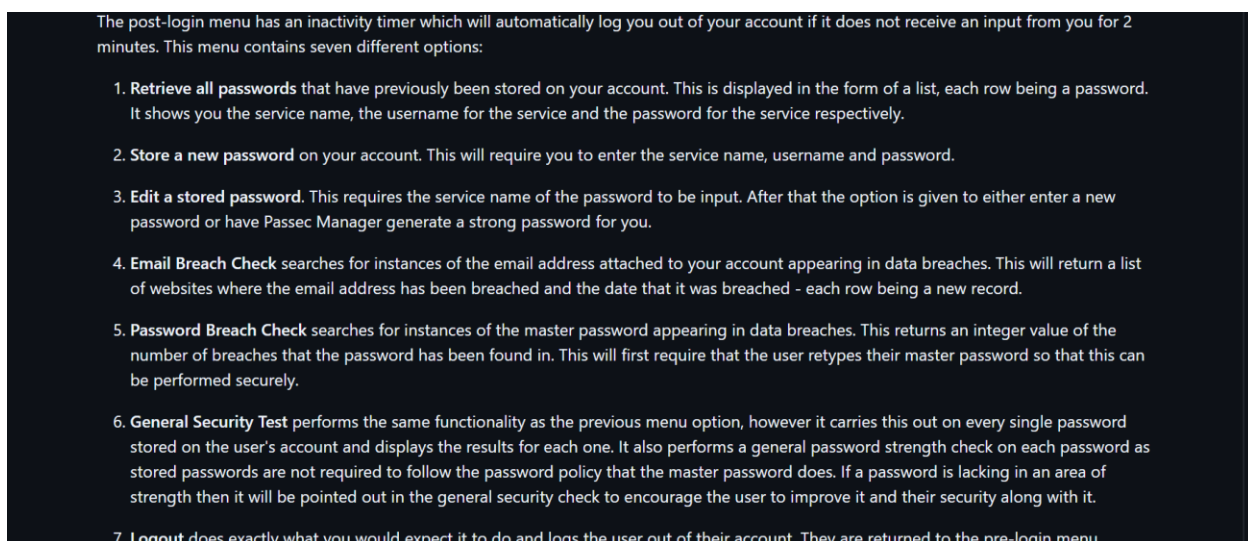


Figure A11 - README graphic (6/6)

APPENDIX B – CODE

```
from userAuth import register, login
from passwordManager import passecMain

# =====TO DO LIST =====
# Email verification - send out an email to prevent users signing up with emails they dont own
# Password reset #
```

```
main( )
```

Figure B1 - Main.py

○ ○ ○

```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi
import os

uri = os.getenv("mongodbURI") #mongodb environmental variable
if not uri:
    print("Error: can't find mongoDB URI")
    exit()

#connect to the server
client = MongoClient(uri, server_api=ServerApi('1'))

db = client["PasseccManager"] #database
usersTable = db["users"] #users table
passwordsTable = db["passwords"] #passwords table

#test connection
try:
    client.admin.command('ping')
    #print("Pinged! You have successfully connected to MongoDB!") #used for testing purposes
except Exception as e:
    print(e)
    exit()
```

Figure B2 – dbconnect.py

```

○○○

from dbconnect import usersTable
from utils import hashPass, passStrengthTester, generatePass
import random
import smtplib
import ssl
import os

#environment variables
appEmail = os.getenv("passecEmail")
appPass = os.getenv("passecEmailPass")

#=====email verification and password reset=====
def genVerifyCode(): #generate recovery code
    code = ""
    for i in range(6):
        randNum = random.randint(48, 57)
        code += chr(randNum)

    return code

def sendEmail(userEmail, code):
    subject = "Passec Password Manager Code"
    body = f"Your code is: {code}\nEnter the code onto the application"
    message = f"Subject: {subject}\n\n{body}"
    context = ssl.create_default_context()
    try:
        with smtplib.SMTP_SSL("smtp.gmail.com", 465, context=context) as server:
            #print("Logging in")
            server.login(appEmail, appPass)
            #print("Sending email")
            server.sendmail(appEmail, userEmail, message)
            #print("Email sent")
    except Exception as e:
        print("Error sending email: ", e)

def verifyEmail(userEmail):
    code = genVerifyCode()
    sendEmail(userEmail, code)

    attempts = 0
    while attempts < 3:
        inputCode = input("Input the verification code sent to your email address: ")
        if inputCode == code:
            print("Email Verified!")
            return True
        else:
            attempts += 1
            print("Incorrect code, try again")

    print("Verification failed, too many incorrect codes")
    return False

```

Figure B3 – userAuth.py (1/3)

```

def recoverPassword():
    userEmail = input("Email address: ")
    if not usersTable.find_one({"email": userEmail}):
        print("Email address not found")
        return False

    code = genVerifyCode()
    sendEmail(userEmail, code)

    attempts = 0
    while attempts < 3:
        inputCode = input("Input the recovery code sent to your email address: ")
        if inputCode == code:
            print("Success, now enter a new password")
            print("PasssecManager requires the user to have a strong password in order to securely hold the other passwords.\n---=<{Password Policy}>=---\n1. Must be 8 or more Characters long\n2. Must have at least 1 special character (!?@#%)\n3. Must contain at least one uppercase character")
            password = None
            while password is None:
                print("Enter a new password, if you want us to automatically generate a strong one then leave the field empty")
                tempPassword = input("Enter a new password: ")

                if tempPassword == "":
                    tempPassword = generatePass(16)

                if not passStrengthTester(tempPassword):
                    print("Password not strong enough, follow the policy above.")
                else:
                    password = tempPassword

            print(f"Save your password: {password}")
            usersTable.update_one(
                {"email": userEmail},
                {"$set": {"password": hashPass(password)}}
            )
            print("Password successfully updated")

            return True
        else:
            attempts += 1
            print("Incorrect code, try again")

    print("Verification failed, too many incorrect codes")
    return False

```

Figure B4 – userAuth.py (2/3)

```

○ ○ ○

#=====register and login functions=====
#add checks for whether reg details are secure (dont need it for logging in if inputs are sanitized on reg)
def register():
    username = None
    emailAddress = None
    password = None

    while username is None:
        tempUsername = input("Enter a username: ")
        if usersTable.find_one({"username": tempUsername}):
            print("Username already exists, try again.")
        else:
            username = tempUsername

    while emailAddress is None:
        tempEmail = input("Enter a valid email address: ")
        if usersTable.find_one({"email": tempEmail}):
            print("Email address already exists, try again.")
        else:
            if verifyEmail(tempEmail):
                emailAddress = tempEmail
            else:
                print("Invalid Email")

    print("PassManager requires the user to have a strong password in order to securely hold the other passwords.\n-
    -==<{Password Policy}>==--\n1. Must be 8 or more Characters long\n2. Must have at least 1 special character (!?
    @#%)\n3. Must contain at least one uppercase character")
    while password is None:
        print("Enter a new password, if you want us to automatically generate a strong one then leave the field
    empty")
        tempPassword = input("Enter a new password: ")

        if tempPassword == "":
            tempPassword = generatePass(16)

        if not passStrengthTester(tempPassword):
            print("Password not strong enough, follow the policy above.")
        else:
            password = tempPassword

    print(f"Save your password: {password}")

    userData = {"username": username, "email": emailAddress, "password": hashPass(password)}
    usersTable.insert_one(userData)
    print("Account successfully created")
    return True

def login():
    username = input("Enter a username: ")
    password = input("Enter a password: ")

    checkUser = usersTable.find_one({"username": username})

    if checkUser and checkUser["password"] == hashPass(password):
        userID = checkUser["_id"]
        print("Successfully Logged In")
        return username, userID
    else:
        print("Invalid details, try again.")
        return None, None

```

Figure B5 – userAuth.py (3/3)

```

○○○

from dbconnect import usersTable, passwordsTable
from utils import hashPass, checkPass, passStrengthTester, checkPwnedPass, generatePass, checkPwnedEmail, getEmail,
encryptData, decryptData
import time
import threading

loggedIn = True #global loggedIn flag
logoutTimer = None #initiate global variable
idleTime = 120 #idle time of 2 minutes

def autoLogout():
    global loggedIn
    time.sleep(idleTime)
    if loggedIn:
        print("Session timed out due to inactivity")
        loggedIn = False

def resetTimer():
    global logoutTimer
    if logoutTimer and logoutTimer.is_alive():
        logoutTimer.cancel()
    logoutTimer = threading.Timer(idleTime, autoLogout)
    logoutTimer.daemon = True
    logoutTimer.start()

def passecMain(username, userID):
    global loggedIn
    while loggedIn:
        resetTimer()
        print(f"\n\n\nPasec Password Manager - Logged in as {username}, {userID}")

        print("\nWhat would you like to do?\n")
        "\n1. Retrieve all passwords"
        "\n2. Store a new password"
        "\n3. Edit a stored password"
        "\n4. Have I been PWNED? - Email"
        "\n5. Have I been PWNED? - Password"
        "\n6. General Security Check"
        "\n7. Logout")
        try:
            menuValue = int(input("Input Value Here: "))
            resetTimer() #idleTimer reset after input
        except ValueError:
            print("Invalid Option")
            continue

        match menuValue:
            case 1:
                getPass(userID)
            case 2:
                storePass(userID)
            case 3:
                editPass(userID)
            case 4:
                userEmail = getEmail(userID)
                if userEmail:
                    checkPwnedEmail(userEmail)
            case 5:
                password = input("Retype your password for security reasons: ")
                if checkPass(username, password):
                    print(checkPwnedPass(password))
                else:
                    print("Password incorrect")
            case 6:
                print("Work in progress")
                genSecCheck(userID)
                #generalSecurityCheckFunction - all passwords for breach and strength check
            case 7:
                print("Logging out...")
                if logoutTimer:
                    logoutTimer.cancel()
                loggedIn = False
            case _: #if value is outwith specified options
                print("Invalid Option")

    return True

```

Figure B6 – passwordManager.py (1/2)

○ ○ ○

```
def getPass(userID):
    dataset = passwordsTable.find({"userID": userID})

    for data in dataset:
        plainPass = decryptData(data['serPass'])
        print(f"Service: {data['service']}, Username: {data['username']}, Password: {plainPass}")

def editPass(userID):
    print("Input the service in which you would like to edit the password")
    inService = input("Service name: ")
    data = passwordsTable.find_one({"userID": userID, "service": inService})
    if not data or inService == "":
        print("Service not found in saved passwords")
        return False

    print("Now enter a new password, if you want us to automatically generate a strong one then leave the field empty")
    newPass = input("Enter a new password: ")

    if newPass == "":
        newPass = generatePass(16)
        print(newPass)

    encryptedPass = encryptData(newPass)
    hashedPass = hashPass(newPass)
    newData = {"$set": {"serPass": encryptedPass, "hashedPass": hashedPass}}

    passwordsTable.update_one({"userID": userID, "service": inService}, newData)

    print("Password successfully updated")
    return True

def storePass(userID):
    service = input("Service: ")
    serUser = input("Username: ")
    serPass = input("Password: ")

    hashedPass = hashPass(serPass) #use hashed pass to check for dupes as encryption is not consistent

    if passwordsTable.find_one({"userID": userID, "username": serUser, "service": service, "hashedPass": hashedPass}):
        print("Password already saved")
        return False

    serData = {"userID": userID, "service": service, "username": serUser, "serPass": encryptData(serPass),
    "hashedPass": hashedPass}
    passwordsTable.insert_one(serData)
    print("Password Successfully saved")
    return True

def genSecCheck(userID):
    print("work in progress")

    dataset = passwordsTable.find({"userID": userID})
    i = 1
    for data in dataset:
        plainPass = decryptData(data['serPass'])
        service = data['service']
        print(f"\n{i}. Password for {service} '{plainPass}': {checkPwnedPass(plainPass)}")
        print("--Password Flaws--")
        passVal = passStrengthTester(plainPass)
        if passVal:
            print("Password is secure")
        i+=1
```

Figure B7 - passwordManager.py (2/2)

```

○○○

#from userAuth import checkPass
from dbconnect import usersTable, passwordsTable
from cryptography.fernet import Fernet
import hashlib
import requests
import random
import os

#Hashing/Encryption functions=====
def hashPass(password):
    return hashlib.sha256(password.encode()).hexdigest()

#one time use to generate and store key - maybe make it always check for file and auto generate if file doesnt exist
...
def genKey():
    key = Fernet.generate_key() #use .env variable when improving security
    with open("secret.key", "wb") as keyFile:
        keyFile.write(key)

genKey()
...

def loadKey():
    secretKey = os.getenv("secretKey")
    if secretKey is None:
        print("Couldn't find secret key")
        return None

    return secretKey

key = loadKey()
cipher = Fernet(key)

def encryptData(data: str) -> str:
    return cipher.encrypt(data.encode()).decode()

def decryptData(data: str) -> str:
    return cipher.decrypt(data.encode()).decode()

#Password Functions=====
def checkPwnedPass(password):
    #hashpassword
    hashedPass = hashlib.sha1(password.encode()).hexdigest().upper() #hash to sha1 - consists of 40 characters
    prefix, suffix = hashedPass[:5], hashedPass[5:] #prefix = first 5 characters, suffix = remaining 35 characters
    #send to api using first 5 chars of hash(prefix)
    url = f"https://api.pwnedpasswords.com/range/{prefix}"
    response = requests.get(url) #returns all hash suffixes (remaining characters in potential hashes NOT the final 5
    chars as previously specified)

    if response.status_code == 200: #OK status
        hashes = response.text.splitlines() #split hashes into seperate lines in array
        for line in hashes: #loop through hashes
            hash_suffix, numberOfBreaches = line.split(":") #split at ":" (api returns hashes in the format of the
            hash_suffix:numberOfBreaches)
            if suffix == hash_suffix:
                return f"Password has been found in {numberOfBreaches} breaches"
            return "Password has not been found in breaches"
        else:
            return f"Error: {response.status_code}"

def checkPass(username, password):
    checkUser = usersTable.find_one({"username": username})

    if checkUser and checkUser["password"] == hashPass(password):
        return True
    else:
        return False

```

Figure B8 – utils.py (1/3)

```

○○○

def checkAllPass(userID):
    dataset = passwordsTable.find({"userID": userID})
    i = 1
    for data in dataset:
        plainPass = decryptData(data['serPass'])
        print(f"{i}. {checkPwnedPass(plainPass)}")
        #print(f"Service: {data['service']}, Username: {data['username']}, Password: {plainPass}")

def generatePass(passLen):
    password = ""
    while password == "":
        for i in range(passLen):
            ranVal = random.randint(33, 126)
            password += chr(ranVal)

        if not passStrengthTester(password):
            password = ""

    return password

#Email Functions=====
def formatData(data):
    varFound = data.get("found")
    sources = data.get("sources", [])

    nameArr = []
    dateArr = []

    for source in sources:
        nameArr.append(source["name"])
        dateArr.append(source["date"])

    if varFound > 0:
        print(f"Username or Email Address has been found in {varFound} breaches")
        print("-----=(Breach-Details)=====")
        for i in range(len(nameArr)):
            print(f"{i+1} Website: {nameArr[i]}, Date: {dateArr[i]}")
    else:
        print("Username or Email was not found in any breaches")

    return nameArr, dateArr

# handle initial text (if success == True: continue)
# then split domain and data into separate arrays so that domain[i] and date[i] match
# Structure - initial text - fields[ip2, address, dob, username, ip1, city, state, name, last_name, phone, ip,
country, profile_name, password, zip, first_name, origin] - sources[name, date]
# var = number of {} for raw text
# nameArr[*var, dateArr[*var#

def checkPwnedEmail(userAcc):
    hashAcc = hashlib.sha256(userAcc.encode()).hexdigest()
    #print("Hash: " + hashAcc)
    response = requests.get(f'https://leakcheck.io/api/public?check={hashAcc}')

    if response.status_code == 200:
        #pprint.pprint(f"Response: {response.text}")
        #data = json.loads(response.text)
        formatData(response.json()) #sends json object to formatdata
    else:
        print(f"Error: {response.status_code}")

def getEmail(userID):
    checkEmail = usersTable.find_one({"_id": userID})

    if checkEmail:
        return checkEmail["email"]
    else:
        print("Error: could not find userID")
        return None

```

Figure B9 – utils.py (2/3)

○ ○ ○

```
#string search function
#for uppercase get an ascii value for each char and check it is within a certain name
#returns true if password meets string search requirements and false if it doesnt
def searchString(password, passLen):
    spChar = "!?@#%"
    spFlag = False #set true if special char found
    uppercaseFlag = False #set true if uppercase char found

    for i in range(passLen):
        if 'A' <= password[i] <= 'Z':
            uppercaseFlag = True
        for x in range(len(spChar)):
            if password[i] == spChar[x]:
                spFlag = True

    return uppercaseFlag, spFlag

#Test password Strength - needs a register function
#Returns true if pass is valid, false if pass is invalid
# =====RULES=====
# Must be 8 or more Characters long
# Must have at least 1 special character (!?@#%)
# Must contain at least one uppercase character#
def passStrengthTester(password):
    passIsValid = True
    passLen = len(password)
    uppercaseFlag, spFlag = searchString(password, passLen)

    if passLen < 8:
        print("Password must be 8 or more Characters long")
        passIsValid = False
    if not uppercaseFlag:
        print("Password must have at least 1 uppercase character")
        passIsValid = False
    if not spFlag:
        print("Password must have at least 1 special character (!?@#%)")
        passIsValid = False

    return passIsValid
```

Figure B10 – utils.py (3/3)

○ ○ ○

Passec Password Manager

Description

Passec Manager is a terminal based password manager designed to securely store, retrieve and manage user passwords with built in security fuctions.

Features

Passec Manager includes:

- Account registration and login
- Email verification
- Password reset
- Strong master password enforcement
- Auto generates strong passwords
- Have I been Pwned and Leakcheck API usage - check for user's private information in data breaches
- Inactivity auto logout
- MongoDB database storing account information and user's passwords
- Store, edit and retrieve passwords

Setup Instructions

Note: This program is best run on a Linux machine as it is easier to set environment variables.

1. Install 'pymongo' package

```
```bash
```

```
pip install pymongo
```

```
```
```

2. Setup pymongo database and tables

3. Create a gmail account to send out email verification/password reset codes

- An app password needs to be generated for this rather than the initially set account password

4. Generate an encryption key - there is a function commented out in utils.py, uncomment and run this function once, then save the encryption key for later steps

5. Set Environment Variables

Check which shell your machine is using

```
```bash
```

```
echo $SHELL
```

```
```
```

- 1) If the output is bash then it will look like this:

```
```bash
```

```
usr/bin/bash
```

```
```
```

Add environment variables to shell config file

```
```bash
```

```
nano ~/.bashrc
```

```
```
```

- 2) If the output is zsh then it will look like this:

```
```bash
```

```
usr/bin/zsh
```

```
```
```

Add environment variables to shell config file

```
```bash
```

```
nano ~/.zshrc
```

```
```
```

After opening the shell config file in the text editor, the following environment variables need to be set to the values you got from the previous steps:

```
**File:** `~/.bashrc` or `~/.zshrc`
```

```
```bash
```

```
export mongodbURI="mongodbExampleURI"
```

```
export secretKey="123412341234123412341234123412341234123412341234"
```

```
export passecEmail="example@gmail.com"
```

```
export passecEmailPass="pass pass pass pass"
```

```
```
```

Figure B11 – README.md (1/2)

○ ○ ○

What Makes a Strong Password?

Passec Password Manager attempts to find the middle ground between having a strong secure password and avoiding scaring off users with too many strict rules.

Passec Manager's Password Policy:

- Contains at least 8 or more characters
- Contains at least 1 special character (!?@#%)
- Contains at least one uppercase, lowercase and numerical character

How To Use Passec Password Manager

The application has a fairly linear design therefore the menus are easy to follow. Input prompt for the user asks for either a numerical value (for navigating menu options) or a string value (for entering details to store/authenticate). Despite this, further insight into each feature/function is given in this section.

Pre-Login Menu

The pre-login menu contains four different options:

1. **Register** a new account for the service. This requires input from three different fields: Username, Email Address and Password. Email verification is required for this and is performed by sending a code to the input email address and waiting for the user to input it into Passec Password Manager during the registration phase.
2. **Login** to an existing account on the service. This requires a valid username and password. After inputting them you will be taken to the post login menu which is covered in the next section.
3. **Recover password** by sending a verification code to the email address attached to the user's account. Upon entering the correct code the user will be given the opportunity to either enter a new password or have a strong password generated for them. The user will then be returned to the pre-login menu where they can login into there account using the new password.
4. **Exit** the application - The application simply stops running.

Post-Login Menu

The post-login menu has an inactivity timer which will automatically log you out of your account if it does not receive an input from you for 2 minutes. This menu contains seven different options:

1. **Retrieve all passwords** that have previously been stored on your account. This is displayed in the form of a list, each row being a password. It shows you the service name, the username for the service and the password for the service respectively.
2. **Store a new password** on your account. This will require you to enter the service name, username and password.
3. **Edit a stored password**. This requires the service name of the password to be input. After that the option is given to either enter a new password or have Passec Manager generate a strong password for you.
4. **Email Breach Check** searches for instances of the email address attached to your account appearing in data breaches. This will return a list of websites where the email address has been breached and the date that it was breached - each row being a new record.
5. **Password Breach Check** searches for instances of the master password appearing in data breaches. This returns an integer value of the number of breaches that the password has been found in. This will first require that the user retypes their master password so that this can be performed securely.
6. **General Security Test** performs the same functionality as the previous menu option, however it carries this out on every single password stored on the user's account and displays the results for each one. It also performs a general password strength check on each password as stored passwords are not required to follow the password policy that the master password does. If a password is lacking in an area of strength then it will be pointed out in the general security check to encourage the user to improve it and their security along with it.
7. **Logout** does exactly what you would expect it to do and logs the user out of their account. They are returned to the pre-login menu.

Figure B12 - README.md – (2/2)